

Argentum Online

Manual de Proyecto

Taller de Programación I

Primer Cuatrimestre 2026

Tomas Kopal - 112840

Facundo Brondo - 97640

Maryuris Artiles - 110188

Tomas Apud Mora - 107968

Indice

1. [Introducción](#)
 2. [Organización del equipo](#)
 3. [Comparación con el plan inicial](#)
 4. [Herramientas utilizadas](#)
 5. [Feedback sobre la materia](#)
 6. [Puntos problemáticos](#)
 7. [Errores conocidos](#)
-

1. Introducción

El presente documento constituye el manual de proyecto correspondiente al Trabajo Práctico Final de la materia Taller de Programación (cátedra Veiga), cuyo objetivo fue el desarrollo de un videojuego multijugador inspirado en Argentum Online, implementado en C++20 bajo el estándar POSIX 2008, utilizando las librerías SDL y Qt.

El sistema está compuesto por tres aplicaciones independientes: un cliente gráfico, un servidor y un editor.

El proyecto fue desarrollado por el equipo conformado por:

- Tomas Kopal — área principal: servidor y protocolo.
- Facundo Brondo — área principal: editor, cliente y renderizado.
- Maryuris Artiles — área principal: servidor y persistencia.
- Tomas Apud Mora — área principal: cliente y renderizado.

Una aclaración, si bien estos fueron los roles que se tuvieron al inicio del proyecto y donde cada integrante puso más foco a lo largo del trayecto, en las últimas etapas del mismo, estos fueron rotando.

El presente manual tiene como finalidad documentar la organización interna del equipo, las herramientas utilizadas, las dificultades encontradas durante el desarrollo y una reflexión final sobre las decisiones tomadas a lo largo del proyecto.

2. Organización del equipo.

2.1. Metodología y herramientas de trabajo

El equipo utilizó GitHub como sistema de control de versiones y organización del código. La dinámica de trabajo fue la siguiente:

- Cada integrante trabajaba en su propia rama (branch) para desarrollar las funcionalidades asignadas.
- Al finalizar una tarea, se abría un Pull Request, el cual debía ser revisado y aprobado por al menos otro integrante del equipo antes de hacer el merge a la rama main. Esto permitió mantener un control de calidad sobre el código integrado y reducir errores antes de llegar a la rama principal.
- Las reuniones de coordinación y seguimiento se realizaban por Discord, donde se discutían avances, bloqueos y se replanificaban tareas cuando era necesario.
- El cronograma general fue provisto por la cátedra y usado como guía de planificación interna del equipo.

2.2. Cronograma semanal — Planificado vs. Realizado

Semana	Fecha	Hito / Entregable	Planificado (según cronograma)	Realizado / Observaciones
1	12/05	Introducción	Roles definidos, repo armado, diagrama de clases	Se cumplió con casi todo, excepto con el diagrama de clases, el cual si fue empezado mas no completado y estuvo solo centrado en la logica del servidor.
2	19/05	—	Movimiento cliente-servidor, protocolo base, concurrencia base	El protocolo y la concurrencia base fueron implementados, más no se pudo llegar a hacer lo mismo movimiento cliente-servidor (Se tenía dificultades para establecer la conexión)
3	26/05	—	SDL (sprites, cámara), animaciones, concurrencia terminada, editor terminado	Se logro realizar el Movimiento cliente-servidor, se tenían los sprites seleccionados para las animaciones, sin embargo aun no se tenia implementado lo logica de las mismas por el lado del cliente. Hubo problemas con la concurrencia, no era posible conectar mas de un cliente a la vez. Para este punto se creia que el programa del editor ya estaba finalizado (en lo que respecta a su implementación)
4	02/06	—	QT (login/listado), HUD, gameloop	Se tenia la base para el programa de login-listado, sin embargo aun no se tenia el HUD

Semana	Fecha	Hito / Entregable	Planificado (según cronograma)	Realizado / Observaciones
			optimizado, <code>installer.sh</code> , cheats	ni el gameloop optimizado. El editor se vio modificado para agregar en su implementación, el definir a qué región pertenecía un tile (celda del mapa)
5	09/06	Entrega intermedia	Compilación sin errores de Valgrind, partida jugable, rendimiento, QT/editor terminados	Se entregó instalador funcional, compilación 100% limpia, protocolo binario y concurrencia completos. Sin embargo, la lógica de gameplay quedó mayormente comentada (solo Move se despachaba); no se pudo jugar end-to-end por un bug de cámara que impedía ver al personaje.
6	16/06	—	Ciudades y pueblos (2/2), clanes (1/2), mini-chat (1/2), persistencia (1/2)	Se solucionó el bug de la cámara que impedía ver al personaje. El renderizado de animaciones estaba hecho, el HUD era funcional, por parte del servidor estaba implementado la lógica para recibir y ejecutar comandos básicos del jugador a los NPCs de zonas seguras. (comprar\ vender\ retirar). Se tenía la base para la persistencia, que consistía en que en un hilo aparte se iría guardando en memoria el estado del juego
7	23/06	Entrega final	Todas las features terminadas, sin mocks, sin errores de Valgrind	Se considera que todas las features fueron implementadas. Persistencia completa.

3.Comparación con el plan inicial.

3.1. Protocolo.

Según el cronograma original, el protocolo binario debía quedar con su estructura base definida en la segunda semana del proyecto (DTOs, funciones de conversión, senders/receivers) y consolidarse rápidamente para poder avanzar en paralelo sobre cliente y servidor. En la práctica, el protocolo terminó siendo uno de los componentes que más iteraciones sufrió a lo largo de todo el proyecto, en lugar de quedar "cerrado" desde temprano como se había planificado. Esto se debió a que, a medida que se avanzaba en la lógica tanto del cliente como del servidor, se iban descubriendo campos de información esencial que el protocolo original no contemplaba y que era necesario agregar. Algunos ejemplos concretos:

- Carga del mapa al iniciar sesión: en el diseño inicial no se había previsto cómo ni cuándo el servidor le enviaba al cliente la información del mapa en el momento del login, lo cual obligó a extender el protocolo para incluir este envío.

- Identificadores de apariencia del personaje (`body_id`, `head_id`): se sumaron sobre la marcha al detectar que el cliente necesitaba esta información para poder renderizar correctamente a cada jugador, y no estaba contemplada en los DTOs originales.
- Otros campos similares se fueron agregando de forma reactiva, a medida que la implementación de features (inventario, vestimenta, etc.) revelaba qué información faltaba transmitir entre cliente y servidor.

Esto generó un desvío respecto del plan inicial: en vez de tener el protocolo estabilizado desde la semana 2-3, se trabajó con un protocolo "vivo", que se iba ajustando en paralelo al desarrollo de las features de cliente y servidor. Si bien esto permitió no bloquear el avance de otras partes del proyecto, también implicó:

- Necesidad de mantener sincronizados cliente y servidor cada vez que se modificaba un DTO o un opcode.
- Riesgo de incompatibilidades temporales entre ramas de distintos integrantes si no se coordinaba bien la actualización del protocolo.
- Tiempo adicional invertido en refactors del protocolo que, de haberse anticipado mejor el diseño inicial, podrían haberse evitado.

3.2 Editor.

Otro componente que sufrió modificaciones respecto del diseño original fue el editor de mapas, aunque a diferencia del protocolo —que se fue ajustando de forma continua durante todo el desarrollo— este cambio se dio en una instancia más tardía del proyecto, durante las pruebas con mapas pequeños ya cerca de la entrega final.

El diseño inicial del editor consideraba que un tile debía informar únicamente si era caminable o no a nivel de la capa de Background. Sin embargo, al hacer pruebas concretas con mapas reducidos, se detectó que esta información era insuficiente: también era necesario que el tile indicara si era caminable o no en la capa de Objetos, dato clave para el spawn de NPCs. Sin esta distinción, no había forma de garantizar que dos NPCs no terminaran spawnados en la misma posición, o que un NPC apareciera sobre un objeto que debería bloquear su ubicación.

Esto llevó a extender el formato de datos del editor (y, por consiguiente, el formato que el servidor lee desde TOML) para que cada tile contemplara la caminabilidad de forma independiente por capa, en lugar de una única bandera global. A diferencia del caso del protocolo, este ajuste no fue anticipado durante el desarrollo incremental porque el problema solo se hizo evidente al testear con datos reales (mapas chicos y spawn de NPCs), lo cual refleja una limitación del enfoque de diseño inicial: ciertos requisitos de la lógica de juego (en este caso, del spawn de NPCs) no fueron considerados al definir el modelo de datos del mapa, y solo surgieron al integrar piezas que en el cronograma estaban planificadas para etapas separadas (editor en semanas tempranas, NPCs/criaturas en etapas más avanzadas).

4. Herramientas utilizadas

4.1. Entornos de desarrollo y sistemas operativos.

Integrante	IDE	Sistema operativo / entorno de prueba
Maryuris Artilles	CLion	Windows + WSL

Integrante	IDE	Sistema operativo / entorno de prueba
Tomas Apud Mora	CLion	Windows + WSL
Tomas Kopal	VScode	VM Ubuntu
Facundo Brondo	Completa	Linux nativo

Dado que el proyecto depende de librerías y APIs propias de Linux, los integrantes con Windows como sistema base utilizaron WSL o una máquina virtual para poder compilar y ejecutar el proyecto en un entorno Linux real, mientras que el resto trabajó directamente sobre Linux nativo.

4.2. Build system

- CMake (versión mínima 3.24, estándar C++20): sistema de build principal del proyecto. Define targets separados para taller_client, taller_server, taller_editor, taller_launcher y taller_tests, cada uno habilitable/deshabilitable mediante opciones (TALLER_CLIENT, TALLER_SERVER, etc.).
- Makefile: wrapper sobre CMake para simplificar los comandos más usados (make para compilar, make test para correr los tests, etc).

4.3. Librerías externas

- SDL2 / SDL2_image / SDL2_mixer / SDL2_ttf / libSDL2pp: renderizado gráfico, carga de imágenes, audio y texto del cliente.
- Qt6 (Widgets): utilizado para el editor de mapas y el launcher (login/signup, creación de personaje).
- toml++ (tomlplusplus): librería header-only para parsear los archivos de configuración en formato TOML (usado sobre todo en los archivos de configuracion para el balance del juego, etc)

4.4. Linters, formatters y análisis estático

- clang-format: formatea automáticamente el código (.cpp/.h) según el estilo definido en .clang-format.
- cpplint: linter de estilo, configurado en CPPLINT.cfg.
- cppcheck: análisis estático de código, con soporte para Qt y reglas habilitadas para todo el proyecto (excluyendo la carpeta tests/).

4.5 Instalador propio.

installer.sh: script con menú interactivo que instala las dependencias del sistema, compila el proyecto, e instala binarios en ~/.local/bin, assets en ~/.local/share/argentum y configuración en ~/.config/argentum.

5. Feedback sobre la materia.

La materia nos pareció una experiencia muy útil para acercarnos a un proyecto de software más grande y realista. A diferencia de trabajos prácticos más chicos, este proyecto nos obligó a organizarnos como equipo, dividir responsabilidades, integrar código de distintas personas y mantener una arquitectura relativamente ordenada durante varias semanas. Uno de los aspectos más positivos fue que el trabajo práctico permitió aplicar muchos temas importantes de la materia: programación en C++, uso de sockets, hilos, sincronización, colas bloqueantes, manejo de recursos, arquitectura cliente-servidor y herramientas de build. También nos sirvió para aprender a usar bibliotecas externas, como SDL, Qt y SDL_mixer, y para entender mejor la importancia de la documentación y los tests. Como punto a mejorar, creemos que al comienzo el alcance del

trabajo puede sentirse bastante grande, especialmente porque hay muchas tecnologías nuevas para aprender al mismo tiempo. Sería útil contar con ejemplos iniciales más guiados sobre cómo organizar un proyecto cliente-servidor o sobre el renderizado de menus y pantallas de juego con SDL. Eso ayudaría en nuestro caso a reducir la incertidumbre inicial y permitiría avanzar antes sobre el aspecto grafico. En general, la materia nos resultó desafiante pero valiosa. El trabajo práctico demandó mucho tiempo, pero nos dejó una experiencia concreta de desarrollo grupal, integración de módulos y resolución de problemas reales de software que podremos aplicar en un futuro.

6. Puntos problematicos.

6.1. Dificultades técnicas.

- Bug de cámara — personaje no visible (detectado en entrega intermedia): uno de los hallazgos más críticos de la entrega intermedia fue que, durante el playtest, no se llegaba a ver ni controlar al personaje en pantalla. La causa: la función que fija el jugador local para que la cámara lo siga nunca llegaba a invocarse, por lo que la cámara quedaba clavada en el origen del mapa. El personaje sí se creaba y dibujaba correctamente del lado del servidor/snapshot, pero quedaba fuera de la vista del jugador. Esto impidió validar end-to-end cualquier mecánica de juego durante esa entrega, ya que ni siquiera se podía ver el resultado del propio movimiento.

-Protocolo binario en constante evolución: como se detalló en la sección anterior, el protocolo fue uno de los puntos más problemáticos en términos de planificación: en vez de estabilizarse temprano, se modificó de forma recurrente a lo largo de todo el proyecto a medida que cliente y servidor revelaban información faltante (carga de mapa al login, IDs de apariencia del personaje, etc.). Esto generó fricción constante para mantener sincronizados los DTOs entre ambos lados.

-modelo de datos del mapa insuficiente para spawn de NPCs:el editor y el formato de mapa fueron diseñados inicialmente con una sola bandera de "caminable" por tile, sin distinguir entre capas. Recién en etapas avanzadas, al testear con mapas chicos, se detectó que esto era insuficiente para garantizar un spawn correcto de NPCs (sin solaparse entre sí ni con objetos), lo que obligó a un rediseño tardío del formato de datos. Lógica de gameplay implementada de forma incompleta respecto al protocolo y la arquitectura

Si bien la infraestructura de comunicación (protocolo, concurrencia) avanzó sólida y temprano, la lógica de negocio asociada (combate, inventario, chat, comercio, NPCs) quedó rezagada: existían los comandos y el protocolo para soportarla, pero buena parte de su implementación permanecía como código comentado o no invocado hasta etapas avanzadas del proyecto. Esto generó un desbalance entre la solidez de la base técnica y el estado jugable del producto en la entrega intermedia.

6.2. Dificultades organizacionales

- En ocasiones reducidas conflictos de merge frecuentes por trabajar todos sobre develop/main. (siendo develop alguna rama referente a la logica del server o del cliente).
- Fricción por entorno de desarrollo (WSL): los integrantes que trabajaban con Windows + WSL se encontraron con un problema concreto al ejecutar el cliente, el juego no capturaba/mostraba correctamente el puntero del mouse, lo cual impedía probar cualquier funcionalidad que dependiera de la interacción con el mouse. Esto generó una demora hasta que se investigó y resolvió la configuración necesaria para que WSL pudiera mostrar correctamente el cursor dentro de la ventana

del juego (típicamente relacionado con la configuración del servidor gráfico X11/WSLg usado por WSL para mostrar aplicaciones gráficas).

7. Errores conocidos.

- En el mapa hay un bug, donde ciertas celdas no están tachadas como no caminables (en la caverna), por lo que en ese rango entre paredes, aparecen Criaturas.
- Error en el renderizado de ciertos detalles en el mapa que se ven duplicados o superpuestos entre ellos